

Running LLMs Locally: A Complete Beginner's Landscape

September 2026

Table of Contents

1	Introduction	3
1.1	What This Covers	3
1.2	What This Is Not	4
1.3	Background Assumed	4
1.4	How This Is Structured	4
1.5	Quick Read if You Are in a Hurry	4
2	Foundations: What You Are Actually Running	5
2.1	Model Artifact	5
2.2	Inference Engine	5
2.3	Server Layer	6
2.4	Quantization (The Lever You Will Use Most)	6
2.5	Context Window and KV Cache	7
2.6	Sampling Controls	7
3	Beyond Text: Choose the Model for the Modality	7
3.1	Practical Model Map	7
3.2	What Changed Recently	10
3.3	Specialist Models Worth Knowing	10
3.4	Image Generation Is Not Vision	11
3.5	Multimodal Memory Costs	11
4	Hardware Reality in 2026	11
4.1	CPU-Only Systems	11
4.2	NVIDIA GPU Systems	12
4.3	AMD GPU Systems	12
4.4	Apple Silicon	13
4.5	Multi-GPU and Multi-Node	13
5	Runtime and Tooling Landscape	13
5.1	llama.cpp and llama-server	13
5.2	MLX-LM and MLX-VLM	14
5.3	Ollama	14
5.4	LocalAI	15
5.5	Transformers and Diffusers	15

5.6	vLLM	15
5.7	SGLang	16
5.8	LM Studio and Jan	16
5.9	Open WebUI	17
5.10	Comparison Snapshot	17
6	Containers, Runtime Hygiene, and Why Reproducibility Wins	18
6.1	Why Containers Matter	18
6.2	Docker vs Podman	18
6.3	Image Tags and Drift	19
6.4	GPU Access Notes	19
7	Deployment Blueprints That Work	19
7.1	Blueprint 1: Personal Chat in Under 20 Minutes	19
7.2	Blueprint 2: Local Coding Assistant in VS Code	20
7.3	Blueprint 3: Persistent Local API Service	20
7.4	Blueprint 4: Multi-Modal Local Platform	21
7.5	Blueprint 5: High-Concurrency API for Teams	21
8	Model Selection in 2026	22
8.1	The Selection Order That Works	22
8.2	Workload Shapes	22
8.3	Family-Level Guidance (Practical)	22
8.4	Size and Quantization Pairing	23
8.5	Where to Source Models	24
8.6	Minimal Local Eval Loop	24
9	API Compatibility and Application Design	24
9.1	Keep App Code on OpenAI-Compatible Calls	25
9.2	Guardrails for Tool Calling	25
9.3	RAG Layer Decisions	25
10	Operations: Keep It Stable Over Time	26
10.1	Baseline Checklist	26
10.2	Update Discipline	26
10.3	Observability That Actually Helps	27
10.4	Security and Privacy Reality	27
11	Decision Guide by Constraint	27
11.1	I want the easiest possible local setup	27
11.2	I want maximum low-level control	28
11.3	I need high concurrency for team APIs	28
11.4	I need one local platform for text + embeddings + audio + image	28
11.5	I need image generation or image editing	28
11.6	I need OCR, screenshots, or document understanding	28
11.7	I need local transcription or speech generation	28
11.8	I need local video generation	28

11.9 I want a polished local chat UI	28
11.10I want private assistant workflows through messaging apps	29
12 5-Day Practical Starter Path	29
12.1 Day 1: Run a Local Model	29
12.2 Day 2: Add Browser UI	29
12.3 Day 3: Use the API Directly	29
12.4 Day 4: Containerize and Persist	29
12.5 Day 5: Compare Two Model Families	29
13 Conclusion	30
14 Appendix	30
14.1 Quick Command Cheatsheet	30
14.2 Further Reading	31

From Zero to Your Own Private AI Stack, With Containers

~6,500 words · September 2026

You have used cloud AI tools. They are useful, fast, and easy.

Now you want control.

You want your prompts and documents to stay on your machine. You want predictable cost. You want to wire models into your own tools without waiting on a vendor roadmap. You want to understand what is actually running.

This guide is for that exact move.

It is written for technical beginners to local inference: people who are comfortable in a terminal and can read config files, but have not yet built a local model stack end to end.

Everything here is written as current guidance for 10 September 2026. Runtime features and model catalogs change faster than this document can, so use the linked official documentation to verify commands and compatibility before you standardize a deployment.

1 Introduction

1.1 What This Covers

This is a practical guide to running large language models locally for chat, coding, API workloads, and agent backends. You will learn:

1. The minimum concepts you need to avoid common dead ends.
2. How hardware limits shape model choices.

3. How the major runtime options differ in 2026.
4. Deployment patterns that work in practice.
5. How text, vision, image-generation, speech, audio, and video models differ.
6. How to choose, test, and operate models without guesswork.

1.2 What This Is Not

This is not a training guide. It does not teach full fine-tuning pipelines, distributed pretraining, or benchmark archaeology. It does not try to be a complete reference for every inference engine.

It is a decision-and-deployment guide: enough depth to make good choices, enough implementation detail to run things today. Model names and runtime support move quickly, so the dated links and model cards in this guide matter more than any static ranking.

1.3 Background Assumed

You can use a terminal, install software, and read logs. If you know what a container is and have used Docker or Podman once, you are set. If you have not, you can still follow along, but expect to pause and look up a couple of commands.

1.4 How This Is Structured

The guide follows one flow:

1. Foundations (the concepts that actually matter at runtime)
2. Hardware constraints (what fits and what does not)
3. Inference stack options (what each tool is good at)
4. Deployment blueprints (copyable patterns)
5. Model and operations decisions (how to stay sane over time)

1.5 Quick Read if You Are in a Hurry

If you need one short answer:

1. Start with Ollama if you want speed-to-first-result.
2. Use llama.cpp directly if you want control.
3. Use vLLM or SGLang for heavy concurrent serving.
4. Use LocalAI if you want one local API for many modalities.
5. Use ComfyUI/Diffusers for image and video generation, not a chat model.
6. Keep your application on OpenAI-compatible APIs so you can switch backends later.

2 Foundations: What You Are Actually Running

Most confusion in local LLM work comes from blending three layers together.

Separate them:

1. The model artifact (weights + metadata)
2. The inference engine (token generation runtime)
3. The server or UI layer (APIs, chat interfaces, auth)

When you keep these layers distinct, the ecosystem stops feeling messy.

2.1 Model Artifact

A model is a large set of learned weights plus metadata about tokenizer, architecture, and prompt formatting.

For local inference, you will mostly see:

1. GGUF files for llama.cpp-class runtimes
2. Safetensors/Hugging Face checkpoints for transformer-native and diffusion runtimes
3. MLX-converted weights for Apple Silicon
4. Pipeline components such as a VAE, text encoder, vision encoder, or multimodal projector

GGUF remains the practical default for many local setups because it is portable and quantization-friendly.

It is not the universal format. Image and video generators are commonly distributed as Safetensors components, while Apple-oriented tools often use MLX conversions. A vision-language model may also need a separate vision tower or `mmpoj` file.

2.2 Inference Engine

The engine performs tokenization, forward passes, cache management, and sampling.

At runtime, a single response loop is:

1. Convert text to tokens.
2. Run those tokens through the model.
3. Compute next-token probabilities.

4. Sample one token.
5. Append token and repeat.

This is why memory bandwidth and cache behavior matter so much. Even with good compute hardware, poor memory fit destroys throughput.

2.3 Server Layer

The server exposes the engine over HTTP and handles request structure, batching, and optional auth.

If you keep to OpenAI-compatible endpoints, you can usually swap runtimes with small config changes.

That API stability is what makes local inference practical for real applications, not just demos.

2.4 Quantization (The Lever You Will Use Most)

Quantization reduces weight precision to make models fit smaller hardware.

A simplified mental model:

1. Higher precision: better quality ceiling, more memory, slower on constrained hardware.
2. Lower precision: smaller footprint, faster, but quality loss appears sooner on complex tasks.

Common practical choices:

Quantization	Typical Use
FP16/BF16	Datacenter or high-memory local GPU runs
8-bit	Quality-sensitive local serving with decent VRAM
4-bit	Common local sweet spot for many personal text-model runs
2-3 bit	Extreme memory constraint, quality trade-off is obvious

In the GGUF world, 4-bit variants like Q4_K_M remain a strong llama.cpp baseline for many assistants and coding workflows. They are not a universal optimum: image/video pipelines and accelerator servers often use different formats, and quantization quality depends on the model and hardware.

2.5 Context Window and KV Cache

Context window is how much text the model can attend to at once. KV cache is the memory used to store token history state.

Longer context means larger KV cache. That memory cost can dominate runs that otherwise fit fine.

A practical rule:

1. Pick model size and quantization first.
2. Then increase context until latency and memory stay acceptable.
3. Do not assume max advertised context is affordable on your hardware.

2.6 Sampling Controls

The three settings that matter most:

1. Temperature: randomness.
2. Top-p: probability mass cutoff.
3. Repetition controls: avoid loops and overuse.

For coding and precise factual tasks, run cooler. For brainstorming, increase controlled randomness.

3 Beyond Text: Choose the Model for the Modality

“Local model” no longer means only a chatbot. The important split is between models that *understand* an input and models that *generate* an output. A vision-language model (VLM) reads an image and returns text or structured data. An image-generation model turns text or reference images into pixels. A speech-recognition model turns audio into text. They are different model classes, with different files, runtimes, and hardware requirements.

Do not assume that a model advertised as multimodal can generate every modality. Most multimodal language models accept text plus images, audio, or video and generate text. Image and video generators usually use diffusion or flow-matching pipelines and are operated through tools such as [Diffusers](#), [ComfyUI](#), or [InvokeAI](#), not a chat endpoint.

3.1 Practical Model Map

Workload	Current local		What to expect
	examples	Typical local path	
Text, coding, reasoning, and agents	Qwen3.5 , Gemma 4 , Granite 4.2 , gpt-oss , LFM2.5	llama.cpp, Ollama, vLLM, SGLang, Transformers, MLX-LM	The broadest ecosystem. Size, quantization, and context length still dominate fit.
Image understanding, OCR, charts, and documents	Qwen3-VL , Gemma 4 , Granite Vision , LFM2.5-VL	Transformers, vLLM, SGLang, Ollama vision models, or supported llama.cpp multimodal builds	These models describe and reason about pixels; they do not create finished images. OCR quality depends heavily on resolution and layout.
Text-to-image and image editing	FLUX.2 , Qwen-Image , Stable Diffusion family	Diffusers, ComfyUI, InvokeAI	Treat the pipeline, text encoder, variational autoencoder (VAE), and transformer/diffusion weights as one artifact. VRAM rises quickly with resolution and reference images. Licenses differ by checkpoint.

Workload	Current local examples	Typical local path	What to expect
Speech recognition and alignment	Whisper , faster-whisper , Qwen3-ASR , Granite Speech	faster-whisper/CTranslate2, Transformers, vLLM, or experimental llama.cpp audio support	Usually much smaller and easier to run than a general LLM. Use a dedicated automatic speech recognition (ASR) model when transcription is the job.
Speech synthesis and audio understanding	Qwen3-Omni , XTTS v2 , CosyVoice	Transformers, project-specific runtimes, or LocalAI	Text-to-speech (TTS), voice cloning, and audio captioning have different quality and consent risks. Check speaker-data terms before cloning a voice.
Video understanding and generation	Qwen3-VL , Qwen3-Omni , Wan2.2	vLLM, SGLang, or Transformers for understanding; Diffusers or model code for generation	Video generation is substantially more demanding than image generation. Wan2.2's 5B text-image-to-video path documents 24 GB VRAM at 720p; its larger paths need much more.
Text and multimodal retrieval	EmbeddingGemma , Qwen3-VL-Embedding and Reranker , BGE-family models	Sentence Transformers, Transformers, Ollama, or a vector database	Embeddings produce vectors, not answers. Evaluate retrieval separately from the language model that writes the final response.

3.2 What Changed Recently

The frontier has moved toward smaller specialist models and sparse larger models. Current families such as [Qwen3.5](#), [Granite 4.2](#), and [Gemma 4](#) illustrate the direction: configurable reasoning, native tool calling, multimodal inputs, and long-context variants are becoming normal product features rather than unusual research demos. These are useful examples of why “parameter count” alone is no longer enough: active parameters, modality encoders, context length, and draft models all affect the actual local cost.

Two other changes matter for personal hardware. [LFM2.5-DSpark](#) adds a small draft-model path for speculative decoding, reporting up to $3.18\times$ GPU and $2.87\times$ on-device throughput on its tested workloads. [Qwen3-ASR](#) provides 0.6B and 1.7B speech-recognition models supporting 52 languages and dialects, plus a 0.6B forced-aligner. These are different strategies from buying a larger general-purpose model: reduce the model’s job, then optimize that job directly.

3.3 Specialist Models Worth Knowing

For image generation, [FLUX.2 Klein](#) is the practical small-model branch, while FLUX.2 dev is a much larger 32B option. The Klein 4B checkpoint is aimed at consumer GPUs; the 9B and dev variants have different license terms, so read the exact model license before commercial use. [Qwen-Image](#) and [Qwen-Image-Edit](#) are larger Apache-2.0 image-generation/editing options with strong typography and reference-image control. [HunyuanImage 3.0](#) is a high-end 80B-total/13B-active MoE model, not a desktop recommendation; its license also excludes some territories, including the European Union.

For document work, a general VLM is not always the best tool. [PaddleOCR-VL 1.6](#) and [GLM-OCR](#) are compact 0.9B-class specialists for layout-aware extraction. [olmOCR 2](#) is a heavier 7B option for difficult PDFs, equations, tables, handwriting, and reading order. This is a useful pattern: use a small specialist to turn pixels into reliable structure, then pass that structure to a language model.

For speech, [Whisper large-v3-turbo](#) remains the broad-compatibility default, while [Qwen3-ASR](#) is a newer small multilingual option. [Kokoro-82M](#) is a practical lightweight TTS model; [Qwen3-TTS](#) adds voice design and zero-shot cloning, which makes consent and speaker-data handling part of the deployment decision.

For video, [Wan2.2](#) remains the most approachable open generation path: its TI2V-5B model documents 720p generation on a 24 GB GPU with offloading. [LTX-2.5](#) is a heavier synchronized audio-video option with an official 32 GB minimum. Video generation is still a

workstation workload even when the headline parameter count looks modest.

3.4 Image Generation Is Not Vision

This distinction prevents a common failed setup. A VLM such as Qwen3-VL can inspect a screenshot, read a receipt, or answer questions about a photograph. An image model such as FLUX.2 or Qwen-Image can synthesize or edit pixels. They may share a text encoder or appear in the same UI, but they are not interchangeable. For a private document pipeline, the usual sequence is:

1. Use a VLM or OCR model to extract structure from the document.
2. Use a text model to classify, summarize, or call tools.
3. Use an image model only if the workflow needs a generated or edited image.

The same separation applies to audio and video. Transcribe with ASR, reason over the transcript with a language model, and synthesize speech or video only at the output stage when required.

3.5 Multimodal Memory Costs

Images, audio, and video become tokens or latent features before the language model sees them. A single high-resolution image or a sampled video can therefore consume more context and key-value (KV) cache than its file size suggests. Start with low resolution, short clips, and a small number of images; increase them only after measuring memory and latency. A model that fits text-only may fail once its vision tower, projector, audio encoder, or video loader is enabled.

4 Hardware Reality in 2026

Hardware constraints are still the biggest determinant of user experience.

4.1 CPU-Only Systems

CPU-only inference is valid, but expectations matter.

CPU-only is good for:

1. Small models
2. Batch embedding jobs
3. Offline or low-interactivity pipelines

CPU-only is not ideal for:

1. Large interactive chat at long context
2. Multi-user concurrent serving
3. Tool-heavy agent loops with strict latency needs

Modern llama.cpp builds are excellent on CPU, but a slow token stream is still a slow token stream.

4.2 NVIDIA GPU Systems

NVIDIA remains the easiest path for predictable high-performance local inference because CUDA tooling is mature across runtimes.

Approximate fit guidance for mainstream quantized models:

VRAM	Comfortable Tier
6-8 GB	Small 3B-8B models at 4-bit
12-16 GB	8B-14B class at 4-bit, some 8-bit options
24 GB	30B class at 4-bit in many setups
48 GB+	70B class at 4-bit for serious local serving

Exact fit depends on runtime, context, and architecture. Always leave memory headroom for KV cache and framework overhead.

4.3 AMD GPU Systems

ROCm is now materially better than it was two years ago. Many local users run AMD successfully in 2026, and Vulkan paths are useful for some consumer cards and applications. Intel XPU support is also present in parts of the serving ecosystem.

Still, setup friction can be higher than CUDA depending on distro, card generation, and runtime support level. Backend support is not uniform: a model may work in llama.cpp on ROCm but not in a particular vLLM or SGLang build, or may support image inputs only on one backend.

Use AMD when:

1. Your target runtime documents strong ROCm support.
2. You are willing to spend time validating kernel and driver combinations.

Avoid wishful assumptions. Test your exact card + runtime pair early.

4.4 Apple Silicon

Apple Silicon remains one of the best personal platforms for local inference due to unified memory and strong Metal acceleration.

The key trade-off is that very large models are memory-bandwidth sensitive. You can run them, but throughput can flatten faster than people expect.

For many users, a well-chosen 8B-30B class model on Apple Silicon provides the best quality-to-latency balance. MLX and `mlx-lm/mlx-vlm` now make Apple-native text and selected vision workloads particularly attractive; model-specific conversion still matters.

4.5 Multi-GPU and Multi-Node

Splitting models across multiple GPUs or nodes is possible and increasingly common in hobby homelabs.

Do it only when you need it. Complexity rises quickly:

1. More failure modes
2. Harder reproducibility
3. More debugging surface in networking and scheduling

If one larger GPU solves your problem, it is often cheaper in time than distributed experimentation.

5 Runtime and Tooling Landscape

There are more choices now, but the core categories are stable.

At the 10 September 2026 cutoff, the release pages still showed a fast-moving stack across [Ollama](#), [vLLM](#), [SGLang](#), [MLX](#), and [Transformers](#). This document deliberately does not pin a “current” version: runtime and model compatibility is more important than the newest tag, and a moving version number ages immediately. Read the model recipe and accelerator requirements before upgrading.

5.1 llama.cpp and llama-server

llama.cpp is still the foundational local inference engine for GGUF workflows.

Use it when you want:

1. Fine-grained control over runtime flags
2. Strong CPU performance
3. Portable execution across many hardware backends

`llama-server` gives you an OpenAI-compatible API endpoint plus basic web chat. Its current multimodal path uses `libmtd` and supports image and experimental audio input for selected architectures. These models commonly need two GGUF files: the language model and a matching multimodal projector (`mmproj`). Check the [multimodal support list](#); a normal GGUF file is not automatically vision-capable.

```
# Serve a model pulled from Hugging Face through llama.cpp
llama-server -hf ggml-org/gemma-3-1b-it-GGUF --port 8080
```

```
# Or serve a local GGUF file
```

```
llama-server -m ./model.gguf --host 0.0.0.0 --port 8080
```

If you enjoy tuning and understanding the engine, this remains a top choice.

5.2 MLX-LM and MLX-VLM

[MLX](#) is Apple’s array framework for Apple Silicon. `mlx-lm` is a strong path for text models on Macs, and `mlx-vlm` adds model-specific vision-language support. These tools exploit unified memory and avoid the CUDA assumptions in many server stacks. They are excellent for a Mac laptop or workstation, but model conversion and support are more model-specific than with Transformers.

For image generation, use an MLX-specific implementation only when the model’s repository documents it. Otherwise use a supported Diffusers or ComfyUI pipeline and expect higher memory pressure.

5.3 Ollama

Ollama is still the fastest path from zero to a working local model, but it is no longer text-only in practice. Current releases support selected vision models, image inputs, tools, reasoning controls, structured output, and an OpenAI-compatible Responses API. On Apple Silicon, selected models can use an MLX-backed path.

Use it when you want:

1. Straightforward model pull/run lifecycle
2. Local API with minimal setup
3. Good integration with coding tools and chat frontends

```
# Install on Linux
```

```
curl -fsSL https://ollama.com/install.sh | sh
```

```
# Run a model immediately
```

```
ollama run gemma4
```

```
# Start API service  
ollama serve
```

Ollama keeps improving ergonomics and model catalog experience. It is not the most configurable engine, and its OpenAI-compatible surface is only a subset: base64 image data works, while remote image URLs and some tool controls do not. It remains the most productive default for a single user who wants to try text or vision models quickly.

5.4 LocalAI

LocalAI is a broad local AI platform exposing OpenAI-style interfaces across text, embedding, audio, and image workflows through multiple backends.

Use it when you want:

1. One local API surface for multiple modalities
2. Flexible backend selection
3. Container-first operations

```
# CPU example  
podman run -d --name localai -p 8080:8080 localai/localai:latest  
  
# NVIDIA example (verify current image tag in LocalAI docs)  
podman run -d --name localai -p 8080:8080 --gpus all \  
  localai/localai:latest-gpu-nvidia-cuda-12
```

It is powerful and flexible, but there is more operational surface area than Ollama. It is a useful compatibility layer, not proof that every underlying model has equal support or quality. Validate each modality separately.

5.5 Transformers and Diffusers

[Transformers](#) remains the model-definition and native-inference layer for many VLM, OCR, ASR, and text models. Its newer `transformers serve` command can expose an OpenAI-compatible endpoint for supported models, which is convenient for testing before moving to vLLM or SGLang.

[Diffusers](#) is the corresponding practical layer for image and video generation. It handles pipelines, schedulers, VAEs, adapters, and quantization. Use it for Python integration and reproducibility; use ComfyUI when you want to inspect and iterate on a graph interactively.

5.6 vLLM

vLLM remains one of the leading choices for high-throughput serving with strong batching behavior and efficient cache management. Its supported-model surface now includes text, im-

age, video, and audio inputs for selected architectures, with the exact combination depending on the model and backend.

Use it when you want:

1. Better concurrent throughput than simple single-request loops
2. Production-oriented serving behavior
3. Strong support for modern large-model serving patterns

Python install route

```
uv pip install vllm
vllm serve meta-llama/Llama-3.3-8B-Instruct
```

For local single-user chat, vLLM can be overkill. For multi-user APIs, it is often the right level of machinery. Its main quantization path is Hugging Face-native (for example FP8, GPTQ, AWQ, MXFP4, NVFP4, and quantized KV cache); GGUF support is experimental and requires the separate [vLLM GGUF plugin](#).

5.7 SGLang

SGLang is now a serious option for optimized LLM, multimodal, and diffusion serving, especially where structured generation and advanced decode/scheduling behavior matter. Its support is accelerator- and backend-specific, so verify the model's recipe before treating a claimed feature as portable.

Use it when you want:

1. Competitive serving performance in modern accelerator stacks
2. Advanced control for structured and multimodal output workloads
3. Another mature path beyond vLLM for high-demand deployments

In practice, serious teams often benchmark both vLLM and SGLang for their exact request patterns before committing. For image or video generation, SGLang-Diffusion is a separate path from ordinary chat serving.

5.8 LM Studio and Jan

LM Studio and Jan remain useful GUI-first desktop choices.

Use them when:

1. You want model experimentation without shell-heavy setup
2. You prefer desktop UX over service administration
3. You still want local APIs for integrations

These tools are excellent for exploration and personal workflows, less so for hardened shared server operations.

5.9 Open WebUI

Open WebUI is still the dominant self-hosted chat frontend in this ecosystem. It is becoming an agent frontend as well, with tool approval and richer state handling. Treat it as a versioned application, not an unpinned image.

Use it when you need:

1. Browser chat UI
 2. Multi-user accounts
 3. RAG-style document chat features on top of your backend
- ```
podman run -d -p 3000:8080 \
 --add-host=host.docker.internal:host-gateway \
 -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
 -v open-webui:/app/backend/data \
 ghcr.io/open-webui/open-webui:v0.11.1
```

It does not replace inference backends. It sits in front of them. Check the [release page](#) before copying the version tag, and keep it patched: an earlier 2026 release line had a DNS-rebinding security advisory.

## 5.10 Comparison Snapshot

| Tool      | Easiest Start | High Control | High Throughput | Multi-Modal Scope                 | Typical Persona            |
|-----------|---------------|--------------|-----------------|-----------------------------------|----------------------------|
| Ollama    | Excellent     | Medium       | Medium          | Text + selected vision/tools      | Solo dev, fast setup       |
| llama.cpp | Medium        | Excellent    | Medium          | Text + selected image/audio input | Tuner, systems-minded user |
| LocalAI   | Medium        | High         | Medium-High     | Broad                             | Platform builder           |
| vLLM      | Low           | High         | Excellent       | Text + selected media             | API team                   |

| Tool                     | Easiest Start | High Control  | High Throughput | Multi-Modal Scope                | Typical Persona          |
|--------------------------|---------------|---------------|-----------------|----------------------------------|--------------------------|
| SGLang                   | Low           | High          | Excellent       | Text + media + diffusion         | Performance-focused team |
| MLX-LM + MLX-VLM         | Medium        | High on Apple | Medium          | Text + selected vision models    | Apple Silicon user       |
| Transformers + Diffusers | Medium        | Excellent     | Medium-High     | VLM/ASR + image/video generation | Python integrator        |
| LM Studio / Jan          | Excellent     | Low-Medium    | Low             | Personal use                     | GUI-first user           |
| Open WebUI               | High (as UI)  | N/A           | N/A             | UI layer                         | Team chat frontend       |

## 6 Containers, Runtime Hygiene, and Why Reproducibility Wins

Containerization is still the safest default for Linux server-style local inference.

### 6.1 Why Containers Matter

You isolate dependencies:

1. CUDA/ROCm stacks
2. Python and native library versions
3. Runtime binaries and model-serving flags

This makes rollback possible and debugging less chaotic.

### 6.2 Docker vs Podman

Both are viable. Podman remains attractive for rootless operation and daemonless workflow. Docker still has broader tutorial gravity.

Pick one and standardize across your own docs and scripts. The consistency is more important than the brand.

### 6.3 Image Tags and Drift

Never treat `latest` as a long-term contract.

For personal experimentation, `latest` is fine.

For repeatable environments, pin tags or digests and record:

1. Runtime image
2. Model identifier and revision
3. Key serving flags

That single discipline prevents many “it worked last week” incidents.

### 6.4 GPU Access Notes

GPU passthrough is still the most common deployment failure point.

Checklist:

1. Validate host driver stack first.
2. Validate container toolkit runtime access second.
3. Only then debug model/server flags.

If you invert this order, you lose hours in application-level logs for a host-level problem.

---

## 7 Deployment Blueprints That Work

These are opinionated starting points that fit common goals.

### 7.1 Blueprint 1: Personal Chat in Under 20 Minutes

Use Ollama + optional Open WebUI.  
`curl -fsSL https://ollama.com/install.sh | sh`  
`ollama run gemma4`

Then add Open WebUI if you want browser chat and conversation history.

Who this is for:

1. First local setup

2. Privacy-first personal usage
3. No strict performance constraints

## 7.2 Blueprint 2: Local Coding Assistant in VS Code

Use Ollama as backend and connect via Continue or other OpenAI-compatible extension.

Example ~/.continue/config.yaml:

```
models:
 - name: Local coding chat
 provider: ollama
 model: qwen3-coder:8b
 roles:
 - chat

 - name: Local autocomplete
 provider: ollama
 model: qwen3-coder:1.5b
 roles:
 - autocomplete
```

If tag names differ in your environment, pick the nearest available coder variants from your local registry.

Who this is for:

1. Daily coding assistance
2. Fast edit-feedback loops
3. Strong local privacy boundary

## 7.3 Blueprint 3: Persistent Local API Service

Use llama-server or Ollama in a container with persistent volumes and service management.

For Linux + Podman + user systemd, prefer Quadlet units.

~/.config/containers/systemd/llm-api.container:

```
[Unit]
Description=Local LLM API

[Container]
Image=ghcr.io/ggml-org/llama.cpp:server-cuda
ContainerName=llm-api
PublishPort=8080:8080
AddDevice=nvidia.com/gpu=all
Exec=-hf ggml-org/Meta-Llama-3.3-8B-Instruct-GGUF --host 0.0.0.0 --port 8080

[Install]
WantedBy=default.target
systemctl --user daemon-reload
systemctl --user enable --now llm-api.service
```

Who this is for:

1. Stable local endpoint for tools/scripts
2. Home lab service-style operation
3. Repeatable restarts and updates

## 7.4 Blueprint 4: Multi-Modal Local Platform

Use LocalAI when you want one endpoint across text, embeddings, speech, and image workflows. For serious image/video generation, pair a dedicated Diffusers or ComfyUI service with your language-model API rather than forcing every modality through one backend.

```
podman run -d --name localai -p 8080:8080 \
 --device nvidia.com/gpu=all \
 -v localai-models:/build/models \
 localai/localai:latest-gpu-nvidia-cuda-12
```

Add models incrementally and validate each modality independently before composing full pipelines.

Who this is for:

1. Prototype platform teams
2. Self-hosted private AI stacks
3. Mixed modality requirements

Keep the model boundaries visible: VLM/OCR for perception, ASR/TTS for speech, diffusion/flow pipelines for image and video generation, and an LLM for language reasoning. A single frontend can hide these services, but it cannot remove their different memory, licensing, and failure modes.

## 7.5 Blueprint 5: High-Concurrency API for Teams

Use vLLM or SGLang behind a simple gateway and benchmark request patterns with realistic prompt lengths.

Who this is for:

1. Internal multi-user applications
2. Latency-sensitive APIs under load
3. Throughput-first architecture decisions

Operational baseline:

1. Structured request logging
2. Queue depth and latency metrics
3. Per-model configuration profiles

4. Rollback plan for model/runtime updates
- 

## 8 Model Selection in 2026

Do not choose by hype. Choose by workload.

### 8.1 The Selection Order That Works

Use this sequence:

1. Define the modality and workload shape.
2. Pick a family with a license that fits your use.
3. Pick size for hardware, including encoders and KV cache.
4. Pick a runtime and its supported format.
5. Pick quantization or an official compressed checkpoint.
6. Run short evals.
7. Freeze the baseline.

### 8.2 Workload Shapes

Four broad categories capture most local usage:

1. General assistant chat and summarization
2. Coding and tool-use agent workflows
3. Reasoning-heavy long-context tasks
4. Perception or generation of images, audio, and video

Different families and sizes win in different categories. There is no universal best model.

### 8.3 Family-Level Guidance (Practical)

As of 10 September 2026, a useful text-model shortlist includes:

1. [Qwen3.5](#) for coding, agents, and multimodal work. Larger mixture-of-experts variants are server-class despite their low active-parameter counts; verify the exact artifact and runtime support before standardizing on one.
2. [Gemma 4](#) for compact reasoning, coding, function calling, and image/audio input on selected variants. Google publishes official quantized memory estimates, which are more useful than a parameter count alone.

3. [Granite 4.2](#) for Apache-2.0 licensed reasoning, coding, native tool calling, and long-context workflows in 3B, 8B, and 30B sizes.
4. [Muse Glimmer](#) for a current 30B local-agent design with image input, a quantized footprint under 20 GB, and speculative decoding. At the August release, some optimized integrations were still landing, so verify runtime support before choosing it as a baseline.
5. [Mistral 3 and Ministral 3](#) for efficient Apache-2.0 text-and-vision models; Mistral Small 4 is a much larger 119B-total-parameter hybrid model and is not a normal laptop choice.
6. [gpt-oss-20b/120b](#) for open-weight reasoning and tool use. The native MXFP4 releases target roughly 16 GB and 80 GB memory envelopes respectively, before runtime headroom.
7. [LFM2.5](#) for small local deployments. Its DSpark draft checkpoints demonstrate that speculative decoding can improve throughput without changing the main model’s output.
8. DeepSeek-R1 distilled models for a well-established reasoning baseline; use the smaller Qwen-derived distills when you need a desktop-sized checkpoint.

Older Llama and Mistral families still matter for compatibility, tutorials, and mature quantized tooling. They are not automatically the best choices for a new deployment.

Treat this as a starting shortlist, not a ranking.

## 8.4 Size and Quantization Pairing

Use realistic tiers:

| Hardware Tier                    | Good Starting Pair               |
|----------------------------------|----------------------------------|
| 4-8 GB VRAM / low-memory systems | 3B-8B at 4-bit                   |
| 12-16 GB VRAM                    | 8B-14B at 4-bit, selective 8-bit |
| 24 GB VRAM                       | 14B-30B at 4-bit                 |
| 48 GB+ VRAM                      | 30B-70B class options            |

Start smaller than your maximum fit. Fast feedback usually beats marginal quality gains from oversized models.

Those tiers describe text-model weights. Specialist models can be much smaller: 0.9B OCR models and 0.6B ASR models fit where a general VLM does not. Conversely, a 20B image generator, a vision encoder, or a video pipeline can exceed the memory of a similarly sized text model. For image/video generation, use the model’s own hardware guidance; for example, Wan2.2 documents 24 GB VRAM for its 5B 720p text-image-to-video path.

## 8.5 Where to Source Models

Primary sources remain:

1. Hugging Face Hub for broad model and quantization availability
2. Ollama library for curated pull-and-run simplicity

Before downloading:

1. Check license fit for your use case.
2. Choose instruct/chat variants unless you need base checkpoints.
3. Confirm file format compatibility with your runtime.
4. Record exact model identifiers used in your stack.

For visual and audio models, also record the encoder/projector, pipeline version, resolution or clip length, and any safety or watermarking defaults. For voice cloning and image-generation checkpoints, record the usage restrictions separately from the code license.

## 8.6 Minimal Local Eval Loop

Do not trust one impression prompt. Build a tiny repeatable eval set.

Include 15-30 prompts that represent your real work:

1. One-turn instruction following
2. Multi-turn memory behavior
3. Coding edits or debugging responses
4. Tool-call formatting correctness
5. Domain-specific reasoning checks

Score each candidate for:

1. Accuracy
2. Latency
3. Stability
4. Cost-to-run on your hardware

That small harness pays off immediately.

---

## 9 API Compatibility and Application Design

You protect future flexibility by designing around stable interfaces.

## 9.1 Keep App Code on OpenAI-Compatible Calls

Most local runtimes expose chat-completions-style APIs.

If your app isolates model client config behind environment variables, you can switch backend without rewriting business logic.

The common shape is useful, but compatibility is not identity. Vision requests may require base64 image data, a model-specific content schema, or a separate projector. Audio transcription and text-to-speech normally use different endpoints. Responses APIs, stateful conversations, tool choice, structured output, and remote image URLs also vary by runtime. Build a small adapter around these differences instead of scattering backend-specific assumptions through your application.

Python example:

```
from openai import OpenAI

client = OpenAI(
 base_url="http://localhost:11434/v1",
 api_key="none",
)

response = client.chat.completions.create(
 model="qwen3-coder:8b",
 messages=[
 {"role": "system", "content": "You are a concise coding assistant."},
 {"role": "user", "content": "Explain what a KV cache does."},
],
 temperature=0.2,
)

print(response.choices[0].message.content)
```

## 9.2 Guardrails for Tool Calling

If you are building agents:

1. Validate JSON/tool schemas strictly.
2. Bound retries and tool loops.
3. Log tool calls and outputs with IDs.
4. Separate model mistakes from tool/runtime mistakes in logs.

This saves enormous debugging time once workflows grow beyond toy examples.

## 9.3 RAG Layer Decisions

For private document workflows, RAG is still the standard pattern:

1. Chunk documents.

2. Generate embeddings.
3. Store vectors.
4. Retrieve relevant chunks at query time.
5. Inject context into generation prompts.

Common local embedding options remain practical through Ollama and open embedding models hosted on Hugging Face. The newer [Qwen3-VL embedding and reranker models](#) extend this pattern to shared text-image-video retrieval, while Google's [EmbeddingGemma](#) is a compact text-retrieval option.

Keep retrieval evaluation separate from generation evaluation. Mixing both in one score hides bottlenecks.

For scanned PDFs, do not assume text extraction is enough. A practical multimodal pipeline can use a specialist OCR/document model such as [PaddleOCR-VL](#), [GLM-OCR](#), or [olmOCR](#), then embed the resulting text and layout. Test tables, formulas, handwriting, reading order, and low-quality scans as separate cases.

---

## 10 Operations: Keep It Stable Over Time

Most local deployments fail from drift, not from initial setup.

### 10.1 Baseline Checklist

For each deployed model/runtime combo, record:

1. Runtime image tag or digest
2. Model ID + revision/tag
3. Key inference parameters
4. Context length
5. Hardware target
6. Expected latency envelope

Store this next to your deployment config.

### 10.2 Update Discipline

Use a simple release procedure:

1. Pull new runtime/model in staging profile.

2. Run your mini eval suite.
3. Compare quality and latency against baseline.
4. Promote only if metrics and behavior are acceptable.

Local stacks feel “small,” but this discipline is still worth it.

### **10.3 Observability That Actually Helps**

You do not need enterprise observability to get value.

Start with:

1. Request ID + model ID in logs
2. Prompt token count, output token count, duration
3. Error rate by endpoint and model

Those three signals usually pinpoint where quality or latency regressed.

### **10.4 Security and Privacy Reality**

Local inference improves privacy posture. It does not automatically make your stack secure.

Still secure:

1. API endpoints
2. Frontend auth
3. Tool execution permissions
4. Secrets management
5. Backup handling for stored chats and vectors

If you expose your service over a network, treat it like any other internal API.

---

## **11 Decision Guide by Constraint**

Use this when you need a fast recommendation.

### **11.1 I want the easiest possible local setup**

Use Ollama first.

## **11.2 I want maximum low-level control**

Use llama.cpp directly.

## **11.3 I need high concurrency for team APIs**

Benchmark vLLM and SGLang for your request pattern.

## **11.4 I need one local platform for text + embeddings + audio + image**

Use LocalAI for a single API surface, but validate each modality independently. For best generation workflows, use LocalAI or an LLM server alongside ComfyUI/Diffusers.

## **11.5 I need image generation or image editing**

Start with FLUX.2 Klein 4B or Qwen-Image/Edit if their licenses fit your use. Use ComfyUI for interactive graphs or Diffusers for code. Do not select a VLM just because it accepts images.

## **11.6 I need OCR, screenshots, or document understanding**

Start with a small VLM or OCR specialist such as Qwen3-VL, Gemma 4, PaddleOCR-VL, GLM-OCR, or olmOCR. Test layout, tables, formulas, and reading order—not only plain paragraphs.

## **11.7 I need local transcription or speech generation**

Use Whisper/faster-whisper for broad transcription compatibility, Qwen3-ASR for a newer multilingual ASR option, and Kokoro or Qwen3-TTS for speech synthesis. Voice cloning needs explicit speaker consent and a license review.

## **11.8 I need local video generation**

Start with Wan2.2 TI2V-5B if you have around 24 GB VRAM and can accept slow generation. Larger Wan2.2 or synchronized audio-video models move into workstation and multi-GPU territory. Check territorial restrictions on Tencent models before adopting them.

## **11.9 I want a polished local chat UI**

Use Open WebUI with your backend, or desktop-first tools like LM Studio or Jan.

## 11.10 I want private assistant workflows through messaging apps

Use a gateway layer such as OpenClaw in front of your local inference stack. For a deep setup and hardening walkthrough, see the [OpenClaw Primer](#).

---

## 12 5-Day Practical Starter Path

If you are new and want momentum without chaos, follow this path.

### 12.1 Day 1: Run a Local Model

Install Ollama and chat with a small model.  
`curl -fsSL https://ollama.com/install.sh | sh`  
`ollama run gemma4`

If that tag is not present in your installation, run `ollama list` and choose a currently published small instruct model. Model catalogs change faster than this document.

### 12.2 Day 2: Add Browser UI

Run Open WebUI and connect it to Ollama.

### 12.3 Day 3: Use the API Directly

Send chat-completions requests and confirm you can call your local model from code.

### 12.4 Day 4: Containerize and Persist

Move your runtime into containers with persistent volumes and service startup.

### 12.5 Day 5: Compare Two Model Families

Run your mini eval prompts across two families and pick one baseline model for your real workflow.

At this point, you are no longer experimenting blindly. You have a repeatable local AI workflow.

---

## 13 Conclusion

Running open models locally in 2026 is no longer a niche hobby. It is a practical software capability, and the useful unit is no longer always a language model: it may be an OCR model, an image pipeline, an ASR model, an embedding model, or a video generator.

The ecosystem is broad, but the core strategy is simple:

1. Keep model, engine, and server concerns separate.
2. Start with the simplest runtime that meets your current need.
3. Choose a specialist model when the job is visual, audio, retrieval, or generative media rather than text.
4. Preserve portability through OpenAI-compatible interfaces where they fit, with a thin adapter for modality-specific differences.
5. Treat updates as controlled changes, not ad-hoc swaps.

Do that, and you get the best part of local AI: private, configurable intelligence you can run, test, and evolve on your own terms.

---

## 14 Appendix

### 14.1 Quick Command Cheatsheet

```
=== Ollama ===
Verify current model tags in the Ollama library before pulling.
ollama pull qwen3-coder:8b
ollama run qwen3-coder:8b
ollama list
ollama rm qwen3-coder:8b
ollama serve

=== llama-server ===
llama-server -hf ggml-org/gemma-3-1b-it-GGUF --port 8080
llama-server -m ./model.gguf --host 0.0.0.0 --port 8080

=== Podman + Ollama ===
podman run -d --name ollama -p 11434:11434 \
-v ollama-data:/root/.ollama \
ollama/ollama

=== Podman + Open WebUI ===
podman run -d --name open-webui -p 3000:8080 \
--add-host=host.docker.internal:host-gateway \
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
-v open-webui:/app/backend/data \
ghcr.io/open-webui/open-webui:v0.11.1

=== Podman + LocalAI (NVIDIA) ===
```

```
Verify the current CUDA-specific LocalAI image tag first.
podman run -d --name localai -p 8080:8080 \
 --device nvidia.com/gpu=all \
 -v localai-models:/build/models \
 localai/localai:latest-gpu-nvidia-cuda-12

=== vLLM ===
uv pip install vllm
vllm serve meta-llama/Llama-3.3-8B-Instruct
```

## 14.2 Further Reading

- [llama.cpp](#)
- [Ollama docs](#)
- [LocalAI docs](#)
- [vLLM docs](#)
- [SGLang docs](#)
- [Open WebUI](#)
- [Hugging Face model hub](#)
- [LM Studio](#)
- [Jan](#)
- [Qwen3-VL](#)
- [Qwen3-ASR](#)
- [FLUX.2](#)
- [Wan2.2](#)
- [MLX](#)
- [Hugging Face Diffusers](#)